Expense Tracker with Monthly Analytics

Project team

Yashashvi Awari	1262251730
Parth Dharmik	1262251813
Mrunmaee Tare	1262253240
Drishti Milani	1262252780
Sharayu Pawde	1262254097

Problem Statement

People often fail to track their daily expenses properly, leading to poor spending control. The aim is to develop a simple C-based program that records expenses, stores them using file handling, and provides a monthly category-wise spending analysis.

Core Objectives

Daily Expense Recording

Record expenses across multiple categories including food, travel, shopping, and more with date and amount details.

Permanent Data Storage

Store all expense data permanently using text file handling, ensuring information persists between program sessions.

Monthly Analysis

Analyse spending habits by calculating category-wise totals and generating visual ASCII bar charts for easy comprehension.

System Features

Key Capabilities

The system provides comprehensive expense management through an intuitive interface with powerful analytical tools.

Add Expenses

Input expenses with date, category, and amount details

View Records

Display all saved expenses from the file anytime

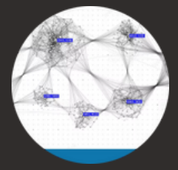
Monthly Summary

Automatic generation of category-wise spending reports

Persistent Storage

All data saved in expenses.txt for future reference

C Programming Concepts Implemented



Structures

Store expense details including date, category, and amount in organised format

Control Flow

Loops and conditional statements to control program execution

File Handling

Functions like fopen, fprintf, and fscanf for reading and writing data

Modular Functions

Separate modules for adding expenses, viewing records, and generating reports

Arrays & Strings

Manage categories and user inputs efficiently

Modules & Working Algorithm

Core Modules

1

Add Expense

Captures user input and stores to file

2

View Expense

Reads and displays data in tabular format

3

Monthly Report

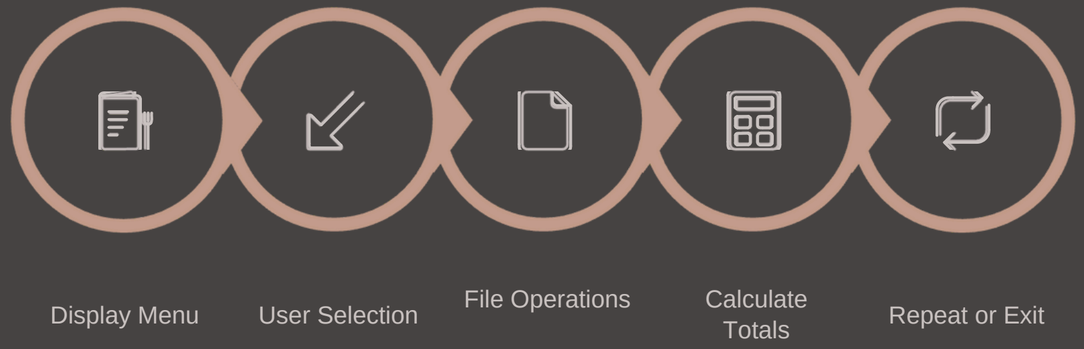
Calculates category-wise totals

4

File Storage

Ensures permanent record keeping

Algorithm Flow



The system follows a simple iterative process where users interact with a menu, perform operations on the file, view analytical results, and continue until exit.

Example UI of Execution:

```
Enter date (DD MM YYYY): 01 02 2026
Enter time (HH MM): 12 00
Enter amount: 1000
Enter expense type without spaces: Groceries
Add another expense? (y/n): y

Enter date (DD MM YYYY): 02 02 2026
Enter time (HH MM): 12 00
Enter amount: 1100
Enter expense type without spaces: Electricity
Add another expense? (y/n): y

Enter date (DD MM YYYY): 03 02 2026
Enter time (HH MM): 12 00
Enter amount: 1200
Enter expense type without spaces: Gas
Add another expense? (y/n): n
```

```
Stored Expenses:

Expense 1
Date: 10/10/2020
Time: 10:10
Amount: 1000.00
Type: bus

Expense 2
Date: 01/02/2026
Time: 12:00
Amount: 1000.00
Type: Groceries

Expense 3
Date: 02/02/2026
Time: 12:00
Amount: 1100.00
Type: Electricity

Expense 4
Date: 03/02/2026
Time: 12:00
Amount: 1200.00
Type: Gas
```

```
1: Total Expenditures
2: Expenditures by month
choose operation to be performed:2
Enter month (MM YYYY):02 2026

Expense 2
Date: 01/02/2026
Time: 12:00
Amount: 1000.00
Type: Groceries

Expense 3
Date: 02/02/2026
Time: 12:00
Amount: 1100.00
Type: Electricity

Expense 4
Date: 03/02/2026
Time: 12:00
Amount: 1200.00
Type: Gas
Total expenses for the month: 3300.00
```

Code in Utilization:

```c
int main(void) {
    loadData();
    int ops = 0;
    int ops2 = 0;
    while (ops != 3) {
        printf("\nWelcome to the Expense Tracker!\n");
        printf("1:Data entry\n");
        printf("2:Data display\n");
        printf("3:Quit and save changes\n");
        printf("choose operation to be performed:");
        scanf("%d", &ops);
        if (ops==1){
            Data_enter();
        }
        else if (ops==2){
            printf("\n1: Total Expenditures\n");
            printf("2: Expenditures by month\n");
            printf("choose operation to be performed:");
            scanf("%d", &ops2);
            if (ops2==1){
                Total_expenses();
            }
            else if (ops2==2){
                Monthly_expenses();
            }
            else {
                printf("Invalid option. Please choose 1, or 2.\n");
            }
        }
        else if (ops==3){
            Quit();
        }
        else {
            printf("Invalid option. Please choose 1, 2, or 3.\n");
        }
    }
    return 0;
}
```

```c
int Total_expenses(){
    printf("\nStored Expenses:\n");
    for (int i = 0; i < count; i++) {
        printf("\nExpense %d\n", i + 1);
        printf("Date: %02d/%02d/%04d\n", expenses[i].day, expenses[i].month, expenses[i].year);
        printf("Time: %02d:%02d\n", expenses[i].hour, expenses[i].minute);
        printf("Amount: %.2f\n", expenses[i].price);
        printf("Type: %s\n", expenses[i].type);
    }
    return 0;
}
// this displays all the data stored in the array
void Monthly_expenses(){
    int month, year;
    float net_expense = 0.0;
    printf("\nEnter month (MM YYYY):");
    scanf("%d %d",&month,&year);
    for (int i = 0; i < count; i++) {
        if (expenses[i].month == month && expenses[i].year == year) {
            printf("\nExpense %d\n", i + 1);
            printf("Date: %02d/%02d/%04d\n", expenses[i].day, expenses[i].month, expenses[i].year);
            printf("Time: %02d:%02d\n", expenses[i].hour, expenses[i].minute);
            printf("Amount: %.2f\n", expenses[i].price);
            printf("Type: %s\n", expenses[i].type);
            net_expense += expenses[i].price;
        }
    }
    printf("Total expenses for the month: %.2f\n", net_expense);
}
// this displays data
int Quit(){
    saveData();
    printf("Exiting the Expense Tracker. Goodbye!\n");
    return 0;
}
```

Advantages, Limitations & Future Scope

Advantages

- Easy to use and understand for beginners
- Practical personal expense management tool
- Demonstrates real-world C programming applications
- Lightweight with simple text file storage

Current Limitations

- Console-based interface only
- Fixed expense categories
- No user authentication
- Limited filtering options

Future Enhancements

- Month-specific filtering capabilities
- Graphical user interface conversion
- Customisable expense categories

> **Conclusion:** The Smart Expense Tracker successfully demonstrates practical implementation of C programming concepts to solve real-life expense management challenges effectively and efficiently.